LECTURE 12

Configuration Space

Robot Motion Planning

Yang (Joyce) Jiao

Adapted from lecture materials by Henrik I. Christensen and Howie Choset
Original contributions from G. D. Hager, Z. Dodds, and Dinesh Mocha

TLDR

Why are we learning configuration space?

Constrained Optimization

$$\min_{x \in D} f(x)$$

Free configuration space

Robot motion planning are essentially constrained optimization

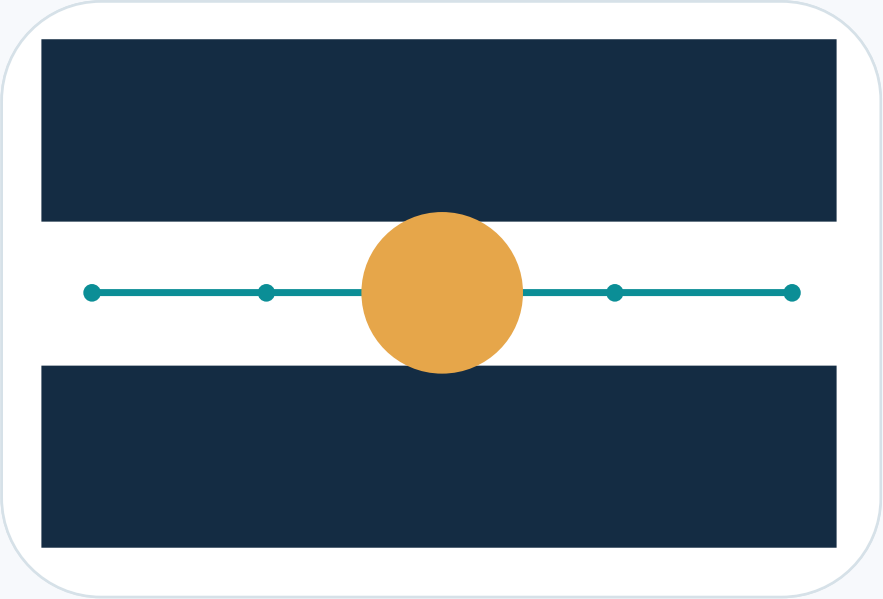
Configuration space helps us to define the constraints.
Path planning defines cost and how we solve the optimization problem

Why is this nontrivial? What may go wrong if we only know the robot position?

MOTIVATION

A robot is not a point

A centerline can avoid obstacles while the body still collides.



Robot center follows a clear line

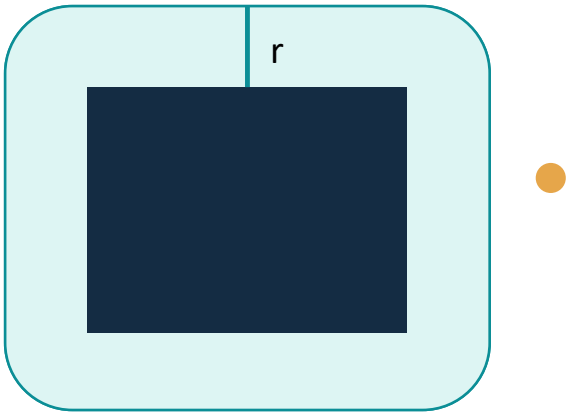
- Point paths ignore body size.
- A reference point alone does not capture collisions.
- We need a representation that includes shape.

Start simple: a translating disk

Move the obstacle, not the robot: represent the disk center as a point.

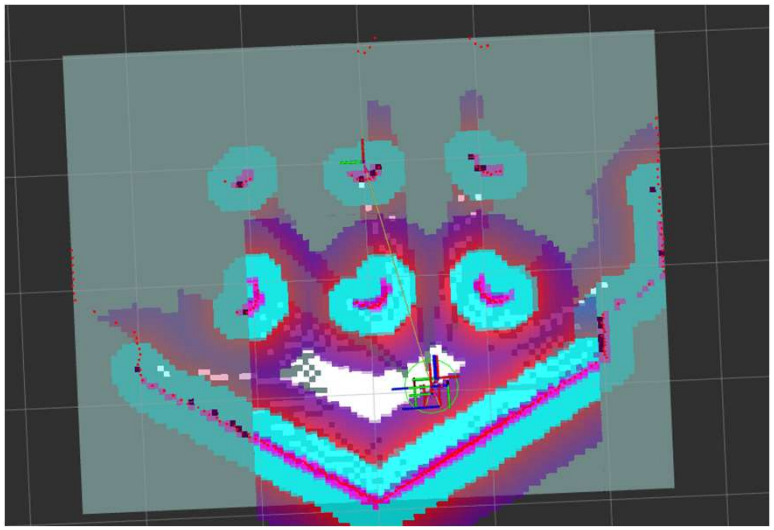


Physical Workspace

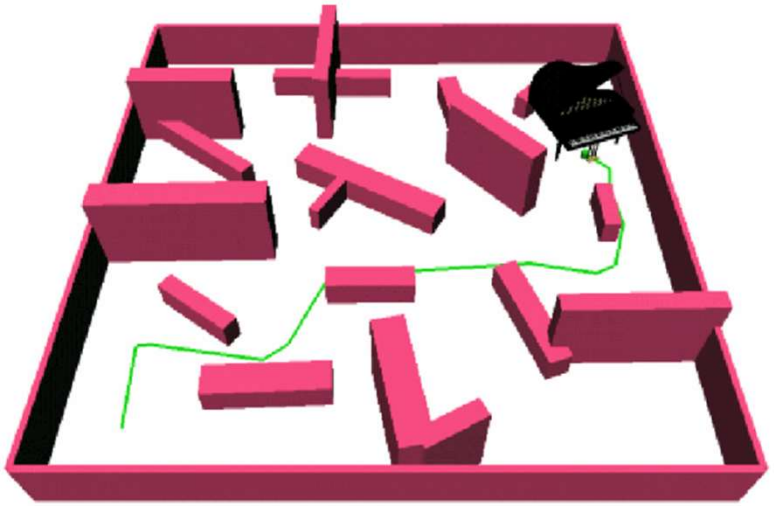


Obstacle Inflation

Are we good to go then?



Yes, it actually works!



Not quite...

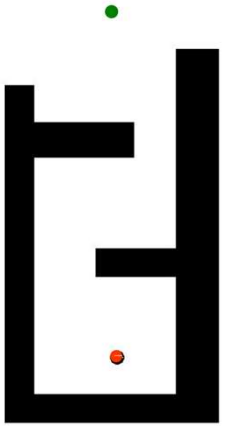
→We need a proper definition of the robot and the environment

ROADMAP

How do we represent all robot motions as paths through a space?

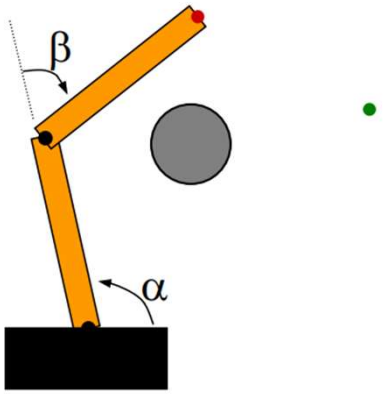
01 Mobile robots

Workspace, complete configurations, and obstacle inflation



02 Manipulators

Joint coordinates, C-space obstacles, and torus topology



03 Planning interface

Free space, continuity, collision checking, planning algorithms

FUNDAMENTAL DEFINITIONS

Workspace vs. configuration space (c-space)

These are different spaces answering different questions.

WORKSPACE W

Physical space where the robot operates (often $\mathbb{R}^2$ or $\mathbb{R}^3$). Workspace obstacles occupy physical regions.

CONFIGURATION SPACE Q

The set of all complete robot configurations. One point $q \in Q$ specifies a full robot posture or pose.

Terminology

For a manipulator, “reachable workspace” usually means attainable end-effector positions or poses.

FUNDAMENTAL DEFINITIONS

A configuration specifies the whole robot

For every q, the occupied body geometry R(q) is determined.

$$q \in Q \quad \rightarrow \quad R(q) \subset W$$

Mobile robot

Point: (x,y) · Rigid planar robot: (x,y,θ)

Robot arm

Joint values: $(\theta_1, \theta_2, \ldots, \theta_n)$

Obstacles can be naturally defined in workspace: $WO \subset W$

Define configuration-space obstacles

Collision is tested between the occupied robot R(q) and obstacles O.

Configuration space obstacle

$$QO = \{ q \in Q \mid R(q) \cap WO \neq \emptyset \}$$

Free configuration space (or just free space)

$$Q_{free} = Q \setminus QO$$

Key transformation

An extended-body collision problem becomes a point-membership problem in Q.

PLANNING PROBLEM

A path is a curve through free configurations

A valid path checks every intermediate configuration, not just its endpoints.

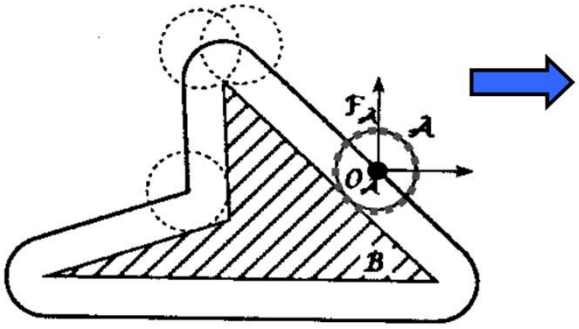
$$c : [0,1] \rightarrow Q_{free}$$



Obstacle inflation for a circular robot

If the robot has radius r, inflate the workspace obstacle by r.

Workspace **C-space**



Inflated obstacle = forbidden centers

Disk of radius r

The inflated boundary consists of offset straight edges and rounded corners.

A center outside it gives geometric clearance.

Polygonal robot: the same construction

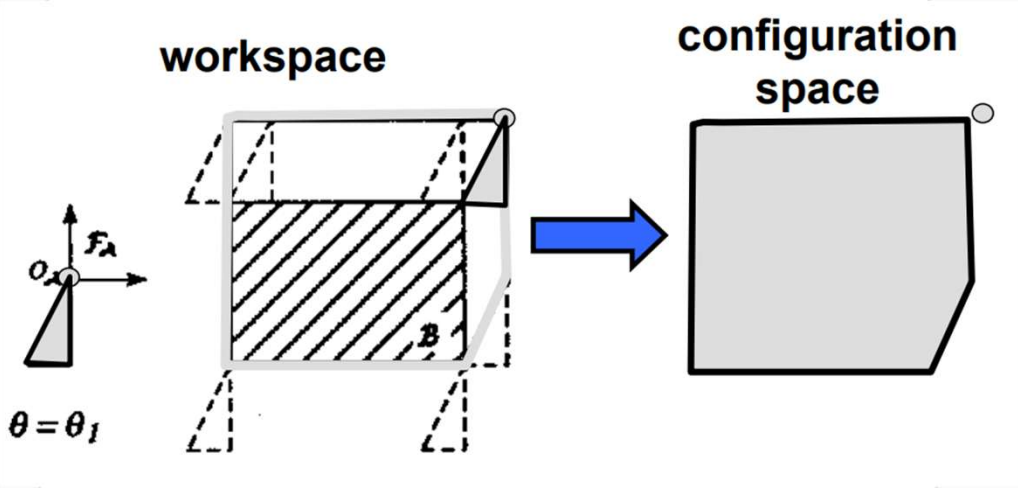
Trace Boundary of Workspace



Obstacle O



Robot R



Forbidden reference-point positions

OBSTACLE CONSTRUCTION

Minkowski sum: add sets of points

A general way to compute forbidden reference-point positions.

Minkowski sum $A \oplus B = \{ a + b \mid a \in A, b \in B \}$

Minkowski difference $A \ominus B = \{ a - b \mid a \in A, b \in B \}$

Geometric meaning

Translate one shape by every point in the other shape; take the union.

Disk case

$O \oplus B_r$ gives the obstacle inflated by a radius-r disk.

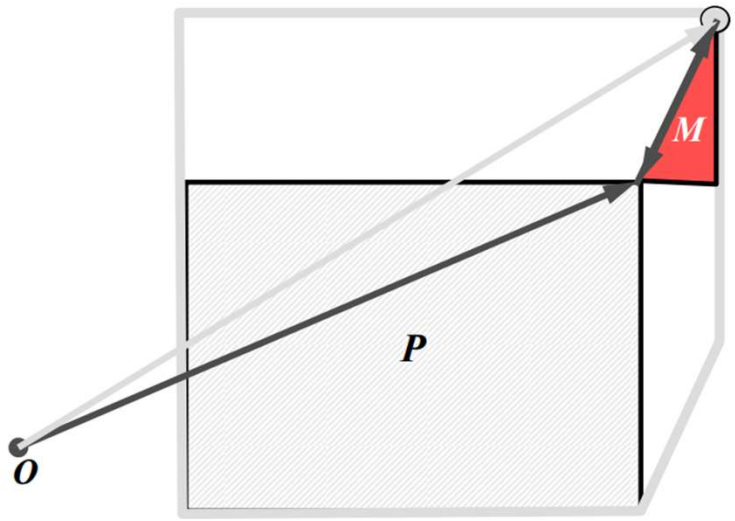
Translating the robot using Minkowski sum

For a translating robot whose shape relative to its reference point is R:

$$Cobs = O \oplus (-R)$$

$$\text{Collision} \Leftrightarrow q + r = o \quad \Leftrightarrow \quad q = o - r$$

for some robot point $r \in R$ and obstacle point $o \in O$



PART 2

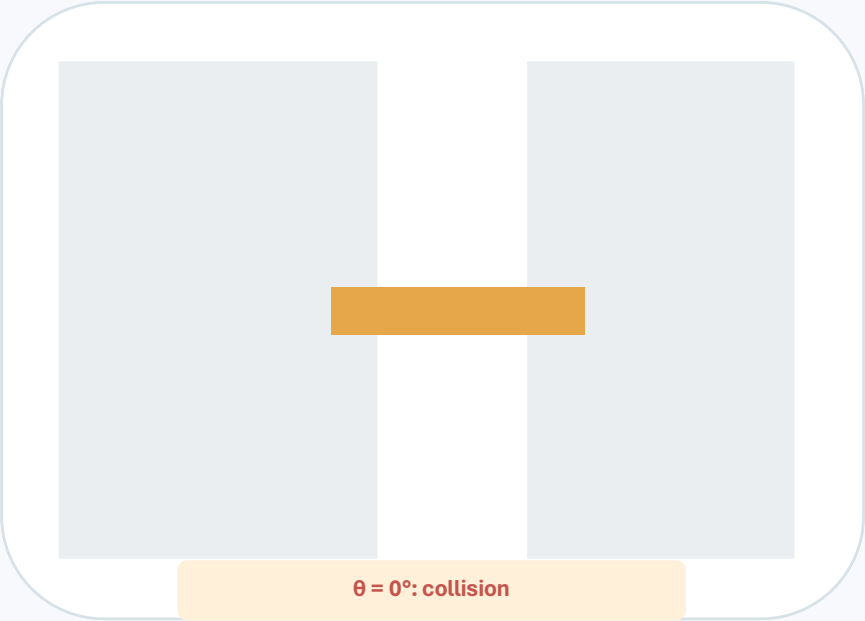
When orientation matters

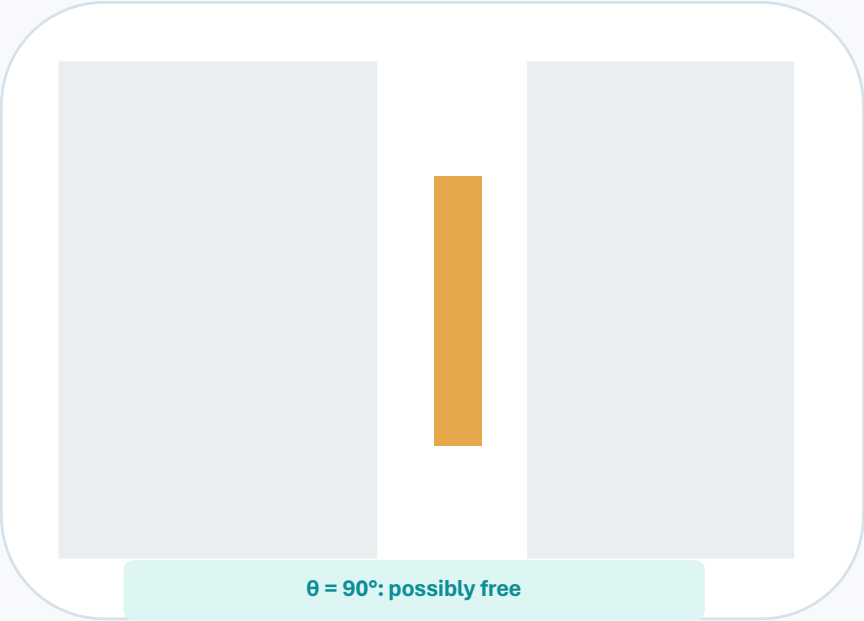
Planar rigid-body motion requires position and rotation.

MOBILE ROBOT WITH ROTATION

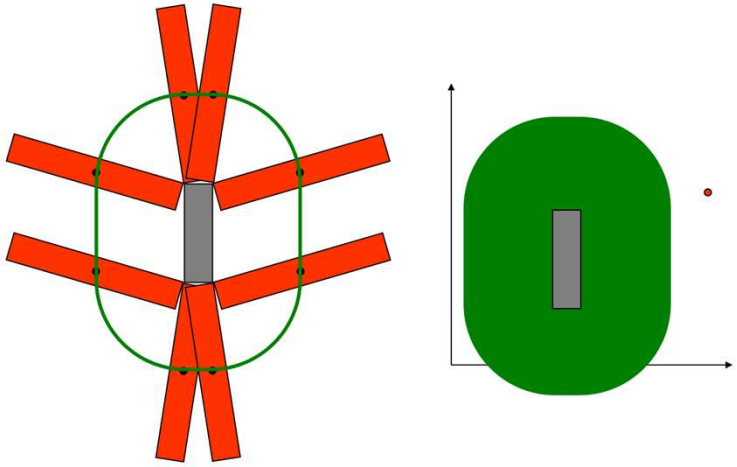
A rectangle may fit only after rotating

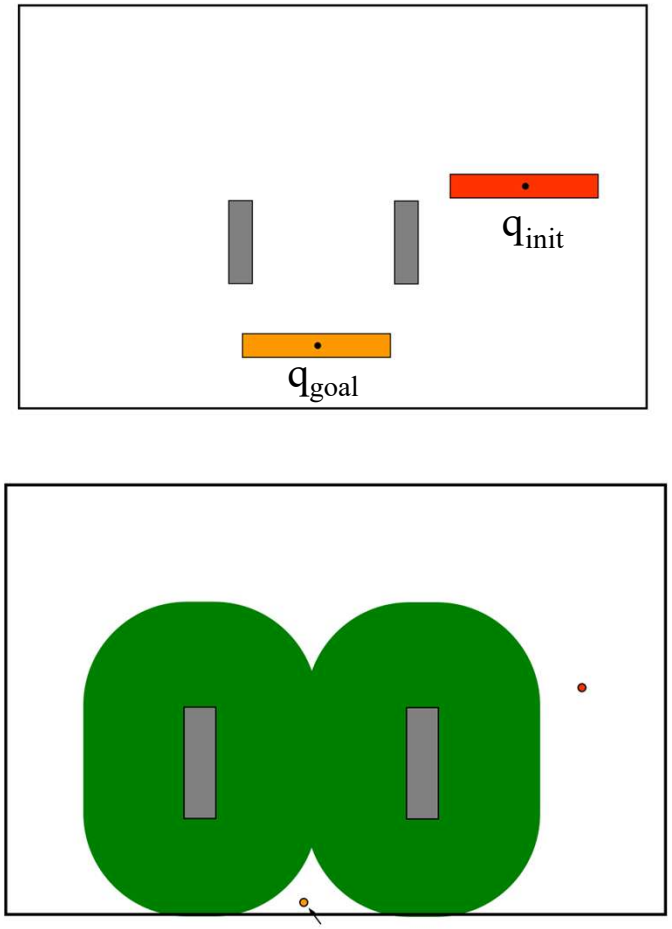
The same position can be free at one heading and in collision at another.



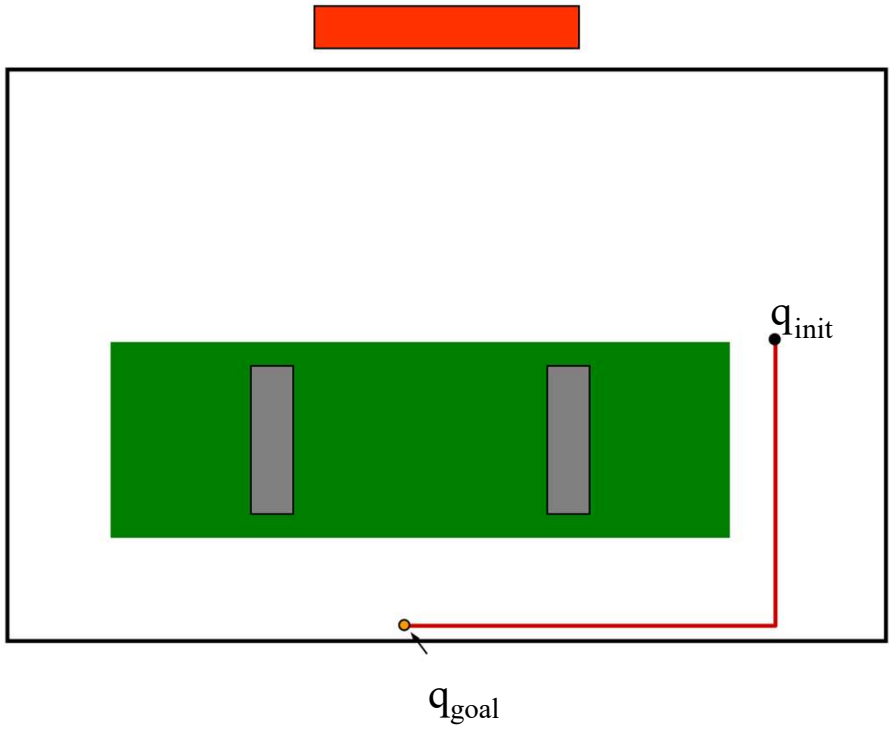


2D Inflation may not be enough

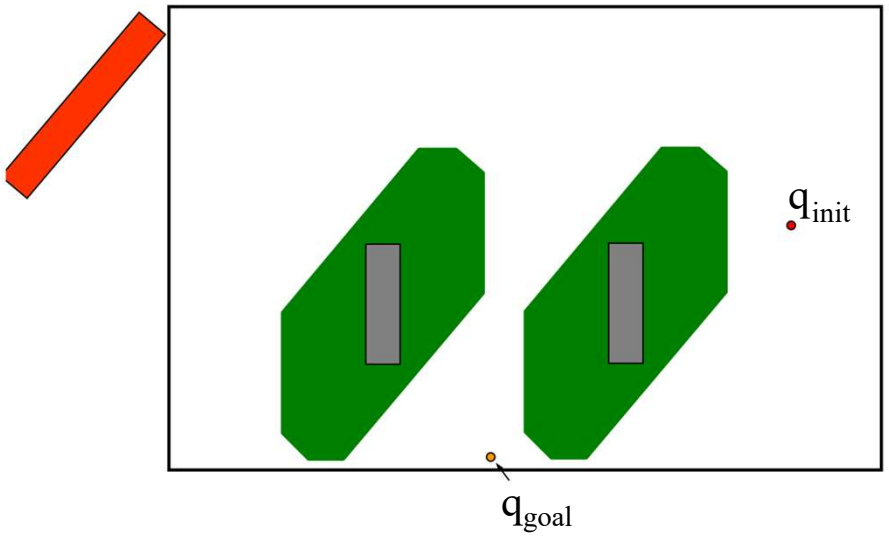




2D Inflation may not be enough

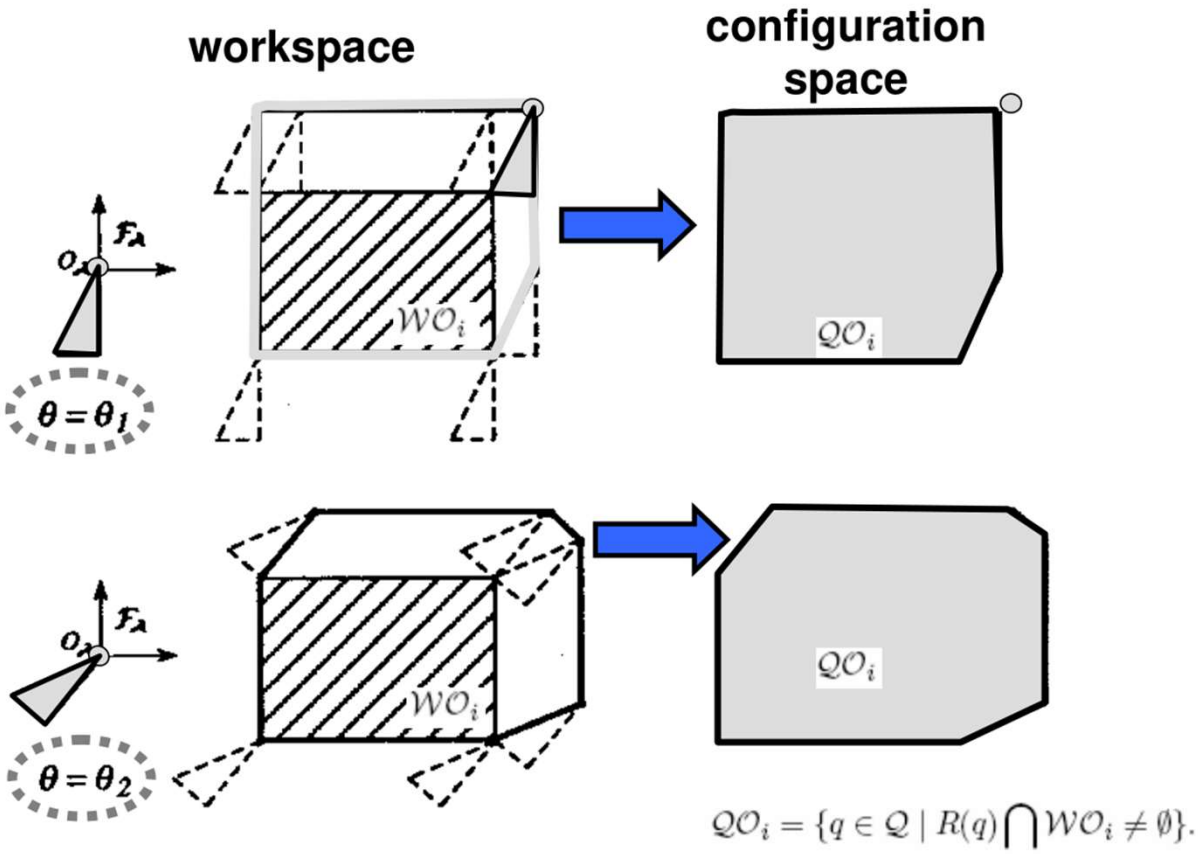


theta=0



theta=45

Same for the polygonal robot



Failure of 2D projection and the alternative

Projecting away θ discards valid orientation-specific motions.

One conservative 2D map

Mark a position forbidden if any possible orientation collides. This can eliminate positions that are actually feasible for some heading.

Full pose space

Keep θ. Each (x,y,θ) is judged independently, so a path can rotate as it passes a tight region.

MOBILE ROBOT WITH ROTATION

A planar rigid-body configuration has 3 DOF

Two translations plus one angular coordinate.

$$q = (x, y, \theta) \in \mathbb{R}^2 \times S^1$$

Position

(x,y) describes translation in the plane.

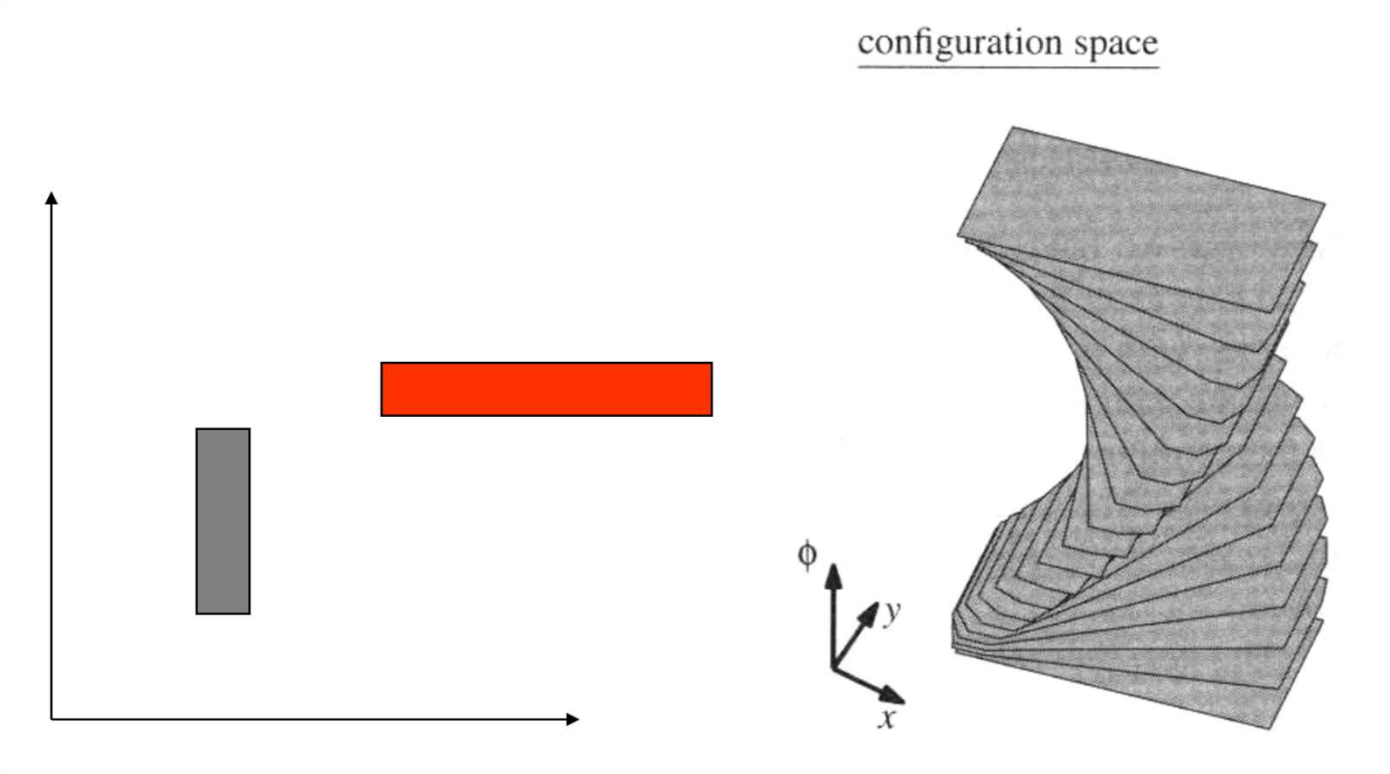
Orientation

θ is periodic: 0° and 360° coincide.

Recall R(q): Q->W

The obstacle becomes a 3D volume

A stack of orientation-dependent 2D forbidden regions.



Think in orientation slices

For fixed θ, compute the forbidden center locations in the plane.

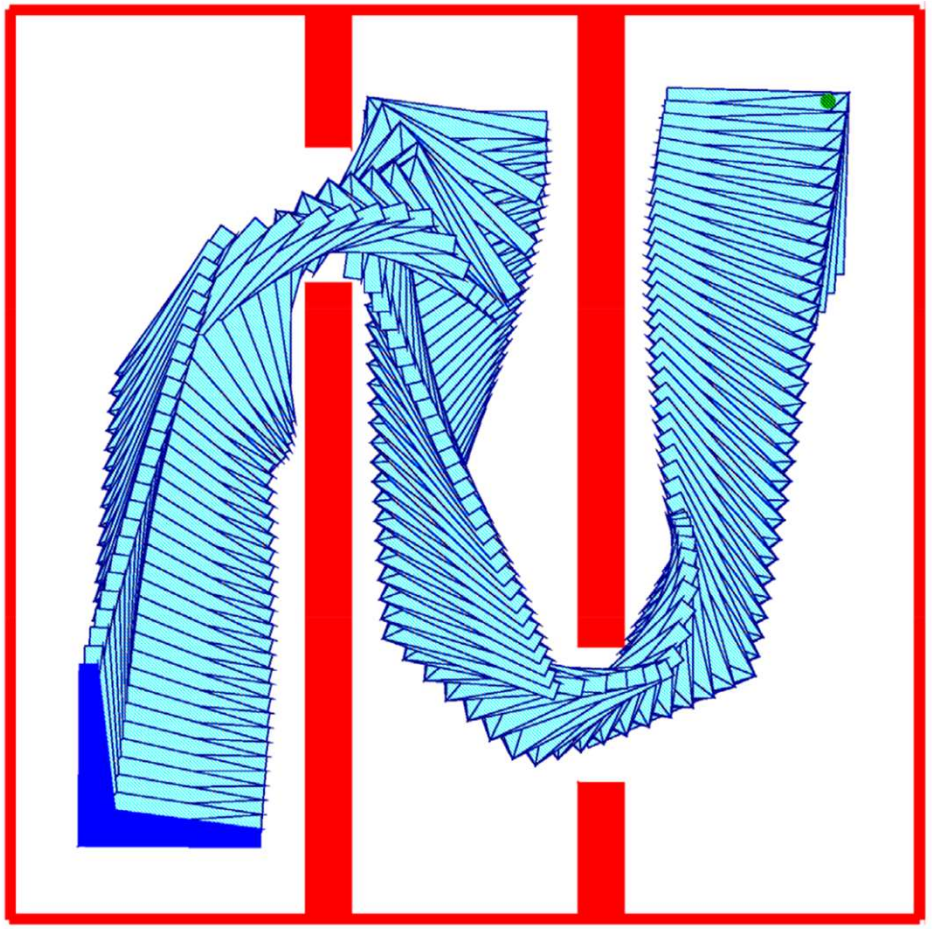
$$Qobs(\theta) = O \oplus (-Rot(\theta)\ R)$$

θ = 0°
one shape

θ = 45°
different shape

θ = 90°
another shape

Moving Piano



Remark: collision-free is not always drivable

Configuration geometry and motion feasibility are related but distinct.

Geometric path

A continuous c(s) entirely inside Qfree. It says nothing by itself about turning radius or velocity limits.

Kinematically feasible path

A carlike robot cannot move directly sideways. Valid paths must also satisfy nonholonomic constraints.

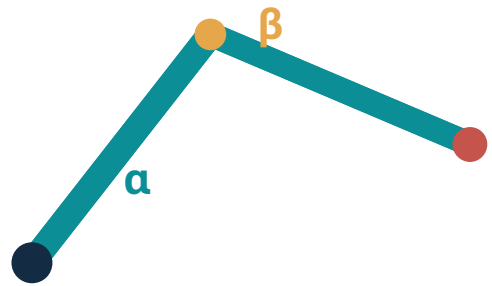
PART 3

Manipulator configuration space

From mobile poses to joint angles and complete arm postures

A two-link arm has two joint coordinates

Joint angles specify the positions of both links.



$$q = (\alpha, \beta)$$

Both link positions are determined by q, given geometry.

Forward kinematics maps Q into workspace

The output is an end-effector position.

$$x = L_1 \cos\alpha + L_2 \cos(\alpha + \beta)$$

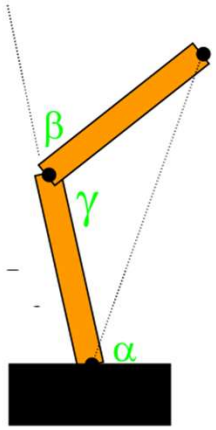
$$y = L_1 \sin\alpha + L_2 \sin(\alpha + \beta)$$

Forward kinematics K: Q → W

For a given joint configuration q, K(q) returns a Cartesian end-effector position.

Note that K(q) is different from R(q)

Inverse kinematics maps W back to Q



$$\gamma = \cos^{-1}\left(\frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}\right)$$

$$\beta = 180 - \gamma$$

$$\alpha = \sin^{-1}\left(\frac{L_2 \sin(\gamma)}{x^2 + y^2}\right) + \tan^{-1}(y/x)$$

atan2(y,x)

PUMA Inverse kinematics



FIGURE 3.17 The Unimation PUMA 560.

```
%
% Solve for theta(1)

r=sqrt(Px^2 + Py^2);
if (n1 == 1),
    theta(1)= atan2(Py,Px) + asin(d3/r);
else
    theta(1)= atan2(Py,Px) + pi - asin(d3/r);
end

%
% Solve for theta(2)

V114= Px*cos(theta(1)) + Py*sin(theta(1));
r=sqrt(V114^2 + Pz^2);
Psi = acos((a2^2-d4^2-a3^2+V114^2+Pz^2)/
           (2.0*a2*r));
theta(2) = atan2(Pz,V114) + n2*Psi;

%
% Solve for theta(3)

num = cos(theta(2))*V114+sin(theta(2))*Pz-a2;
den = cos(theta(2))*Pz - sin(theta(2))*V114;
theta(3) = atan2(a3,d4) - atan2(num, den);
```

```
% Solve for theta(4)

V113 = cos(theta(1))*Ax + sin(theta(1))*Ay;
V323 = cos(theta(1))*Ay - sin(theta(1))*Ax;
V313 = cos(theta(2)+theta(3))*V113 +
       sin(theta(2)+theta(3))*Az;
theta(4) = atan2((n4*V323),(n4*V313));

% Solve for theta(5)

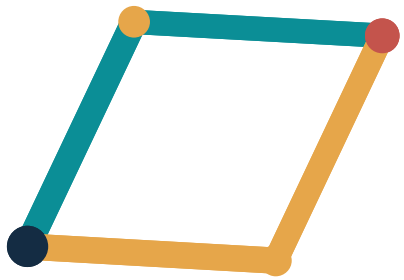
num = -cos(theta(4))*V313 - V323*sin(theta(4));
den = -V113*sin(theta(2)+theta(3)) +
       Az*cos(theta(2)+theta(3));
theta(5) = atan2(num,den);

% Solve for theta(6)

V112 = cos(theta(1))*Ox + sin(theta(1))*Oy;
V132 = sin(theta(1))*Ox - cos(theta(1))*Oy;
V312 = V112*cos(theta(2)+theta(3)) +
       Oz*sin(theta(2)+theta(3));
V332 = -V112*sin(theta(2)+theta(3)) +
        Oz*cos(theta(2)+theta(3));
V412 = V312*cos(theta(4)) - V132*sin(theta(4));
V432 = V312*sin(theta(4)) + V132*cos(theta(4));
num = -V412*cos(theta(5)) - V332*sin(theta(5));
den = - V432;
theta(6) = atan2(num,den);
```

The same end-effector point can have different postures

An IK target does not always specify a safe path or collision-free arm.



Distinct configurations

Planning implication

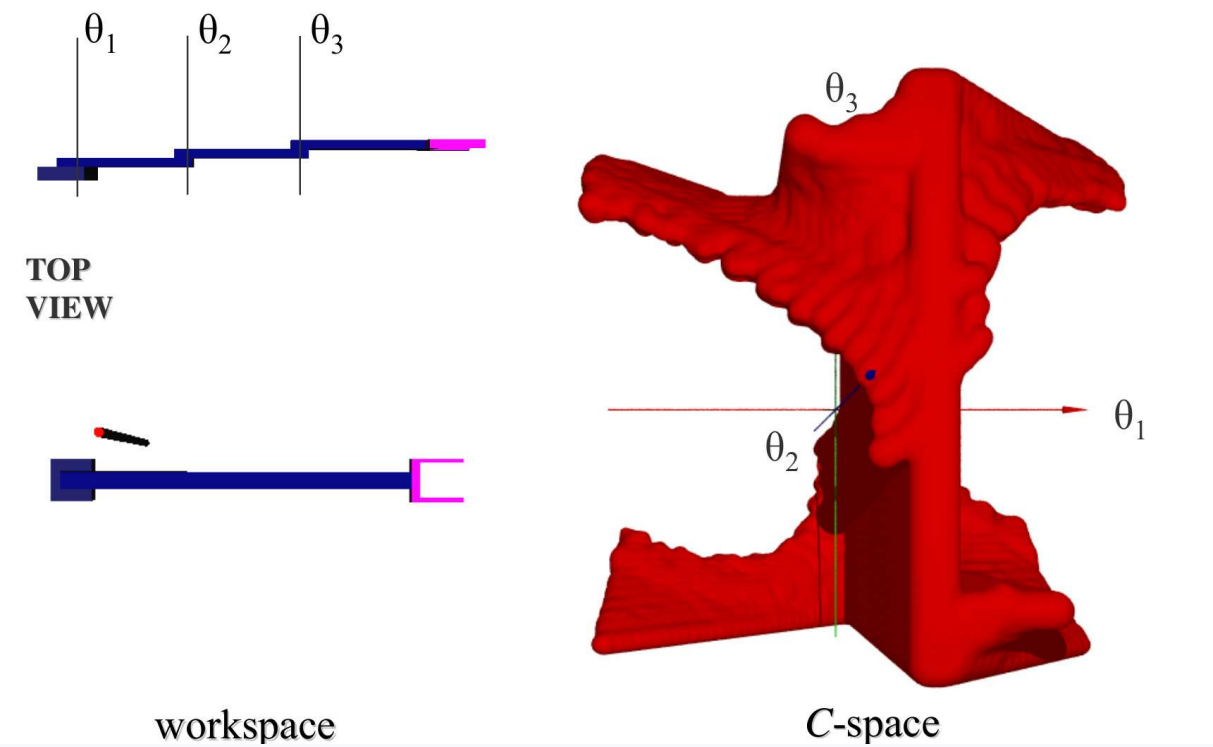
Endpoint goals are often a *set* of acceptable joint configurations.

Collision checking must inspect all links, not just the tip.

Takeaway: IK is nontrivial, but we also have toolbox available

Motion planning searches joint configurations

A curve through joint space generates a sequence of physical arm postures.



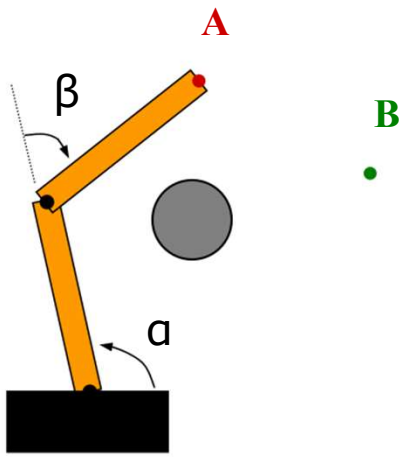
Key idea

A 3-joint arm lives in a 3D joint-coordinate space.

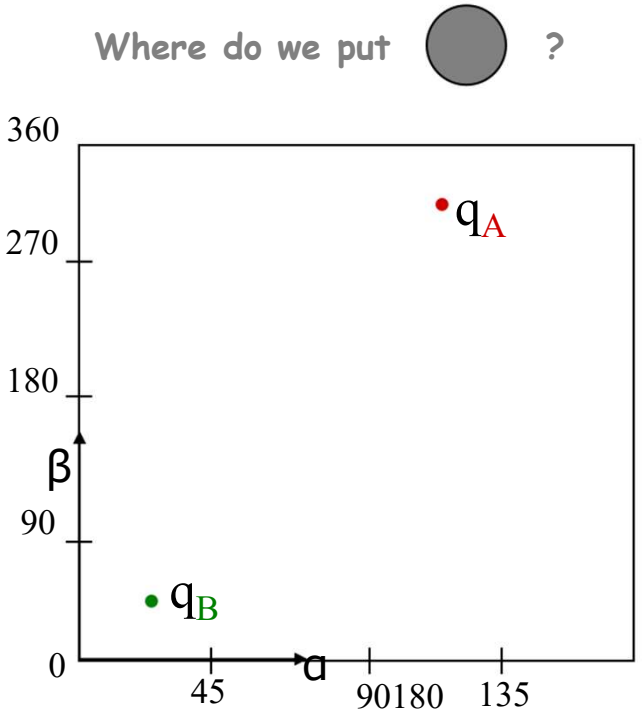
A point there is a full posture.

Translating workspace obstacle into c-space

An IK target does not always specify a safe path or collision-free arm.



An obstacle in the robot's workspace



Torus

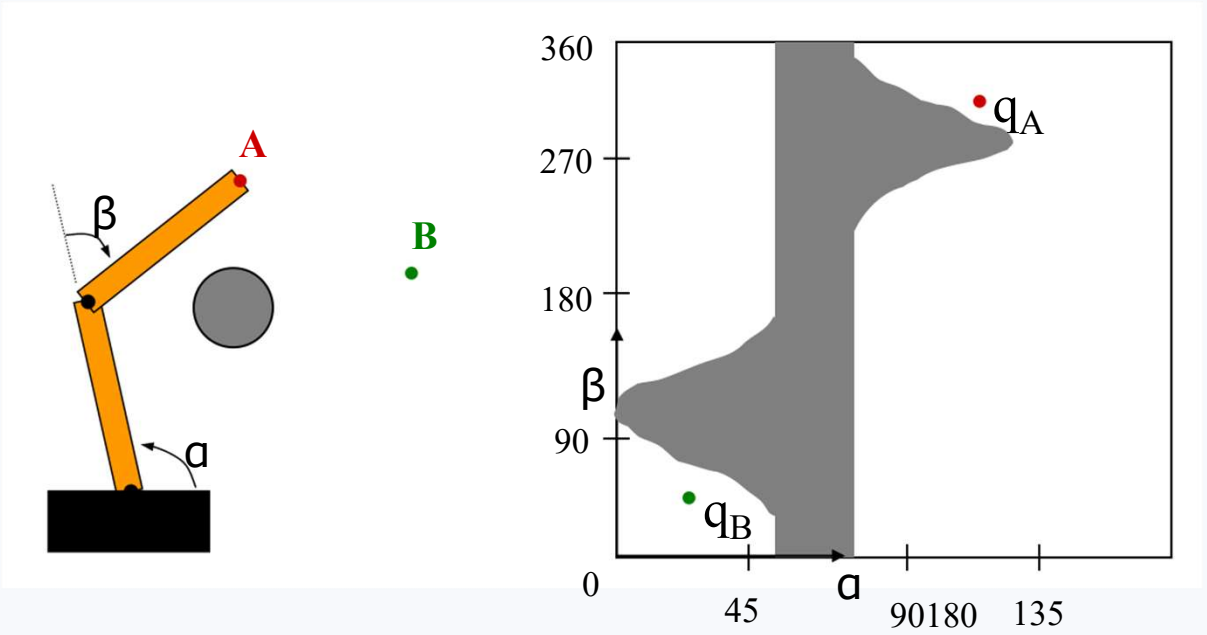
(wraps horizontally and vertically)

Translating workspace obstacle into c-space

Every arm posture intersecting the obstacle is forbidden.

Reference *configuration*

How do we get from **A** to **B** ?



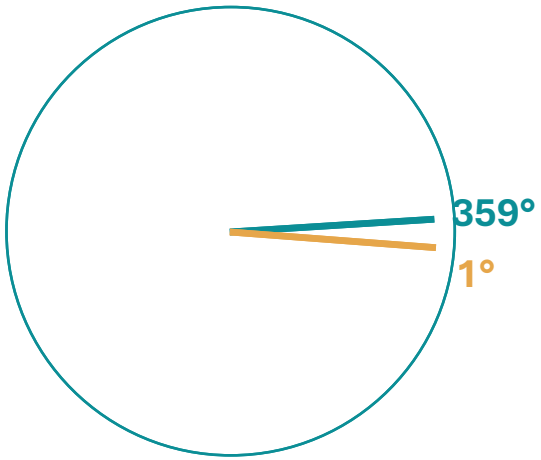
An obstacle in the robot's workspace

The C-space representation of this obstacle…

TOPOLOGY

Angular distance must respect wraparound

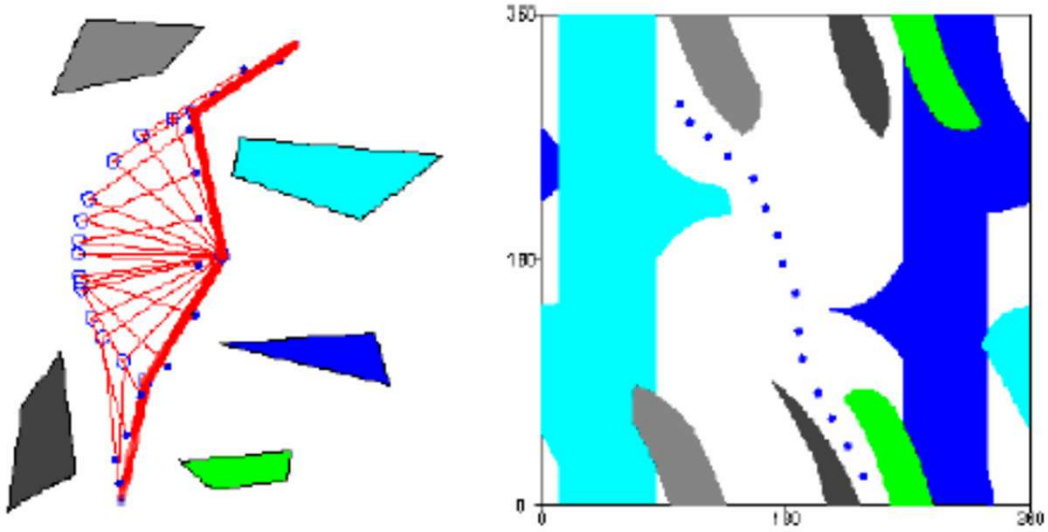
A naive Euclidean difference in coordinates can be misleading.



$$d(359°, 1°) = 2°$$

Not 358°

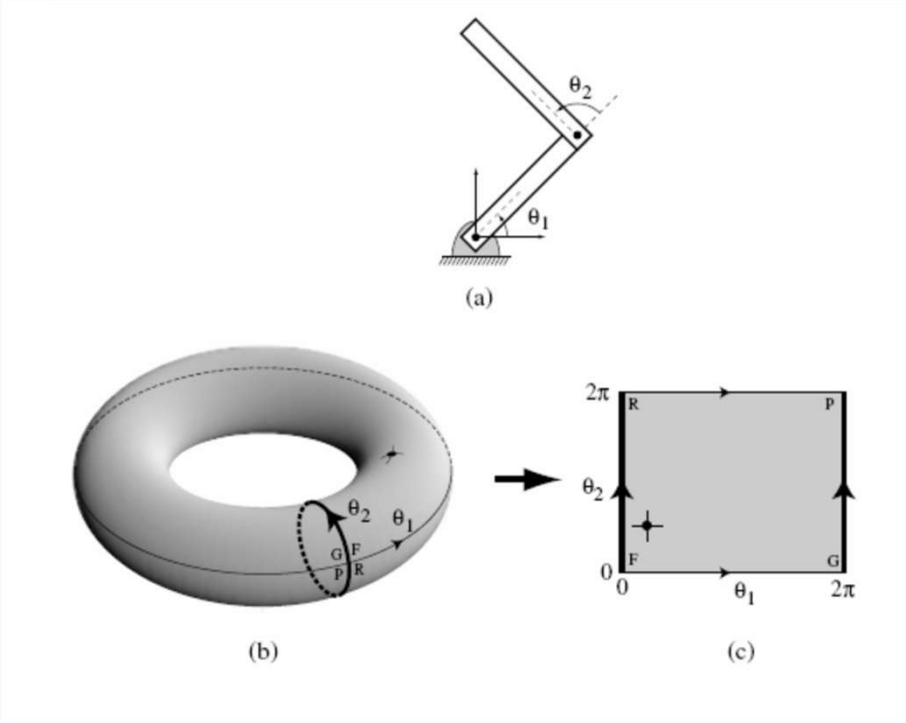
A different angle warping gives a path planning solution



This is because of *topology* difference: joint angles are not cartesian

Two revolute joints form a torus

Unrestricted angles wrap independently.



2R configuration space

$Q = S^1 \times S^1 = T^2$

Opposite edges of the coordinate square are identified.

Joint limits change the permissible configurations

Topology depends on physical joint ranges and obstacles.

Unrestricted joints

If θ_1 and θ_2 both rotate freely, the nominal Q is T^2.

Limited joints

If $\theta_i \in [a_i, b_i]$ without wrap, the allowed joint domain is a box-like subset, further reduced by collisions.

ROBOT ARMS

More joints means more dimensions

Each independent joint coordinate increases the C-space dimension.

2R planar arm

2 DOF
$Q = T^2$

6R industrial arm

6 DOF
$Q = T^6$ (if all unrestricted)

Mobile manipulator

Planar base + arm
$Q \approx SE(2) \times T^n$

PART 4

Topology and manifolds

Why coordinate choices matter to robot motion planning

TOPOLOGY

Dimension does not determine the shape of a space

Spaces of equal dimension can connect differently.

$\mathbb{R}^2$: plane

Two unbounded translational coordinates.

$S^1 \times \mathbb{R}$: cylinder

One periodic angle and one unbounded coordinate.

T^2: torus

Two independently periodic coordinates.

A manifold looks Euclidean locally

But one global coordinate chart may not cover it smoothly.

Local coordinates

Small neighborhoods of a smooth circle look like a line; sphere and torus neighborhoods look like small planar patches.

Global seams

Angles require wraparound. Latitude–longitude coordinates have singularities at the poles.

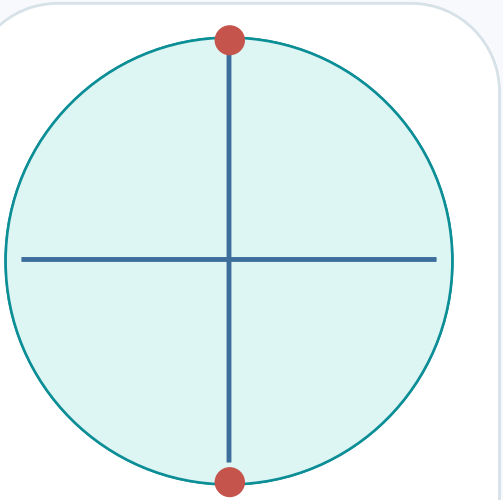
Common robot configuration spaces

A reference table connecting robot models to spaces and dimension.

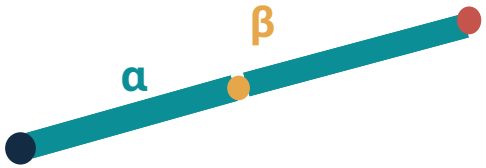
ROBOT	C-SPACE Topology	DIM
Point in 2D plane or 3D space	$\mathbb{R}^2$ or $\mathbb{R}^3$	2 or 3
Planar rigid body	SE(2) or $\mathbb{R}^2 \times S^1$	3
Free rigid body in 3D	SE(3) or $\mathbb{R}^3 \times SO(3)$	6
n unrestricted revolute joints	T^n	n
Planar mobile base + nR arm	$SE(2) \times T^n$	3+n

Chart singularities vs physical singularities

A coordinate representation may fail where the underlying geometry is fine.



North/south poles



Pyhsical Singularity

Dimension of Null space changes.
Can only generate velocity at a certain direction
(kinematics singularity).

Latitude and longitude

At the poles, longitude is not uniquely defined.
The sphere is still smooth; a different chart can represent that neighborhood.

PART 5

What does a planner need?

Represent configurations → query collisions → find a path

Inputs and output of geometric path planning

The problem is algorithmic once Qfree and configurations are defined.

INPUTS

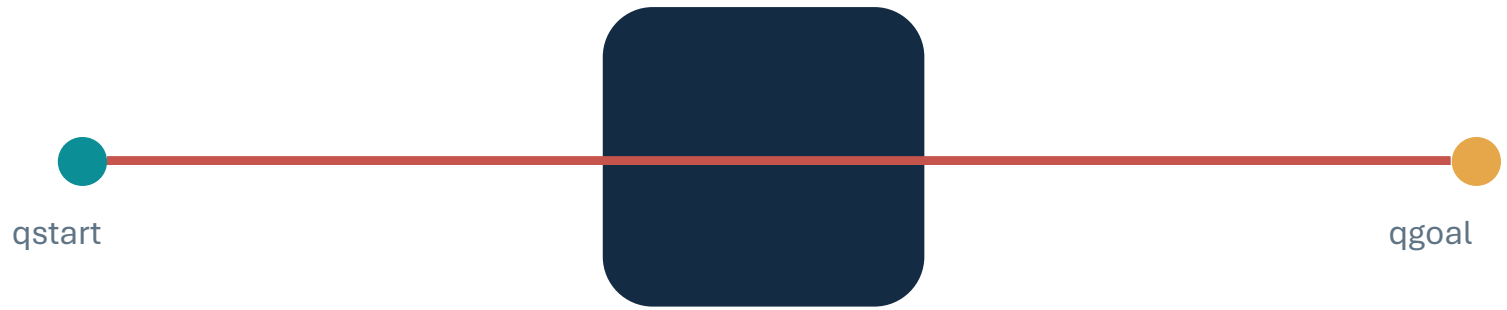
Configuration space Q
Robot and obstacle geometry
Start qstart and goal qgoal
Collision checker

OUTPUT

A continuous c:[0,1]→Qfree
With c(0)=qstart
And c(1)=qgoal
Or an appropriate failure result

A collision-free endpoint pair is not a valid edge

Local connections must be checked continuously or at sufficient resolution.



Unsafe straight interpolation

Three algorithmic questions for next lecture

Different motion planners make different trade-offs.

- How do we represent or explore a continuous, high-dimensional Q?
- How do we test collision-free connections efficiently?
- When can we guarantee a path, or improve its quality?

Constrained Optimization

$$\min_{x \in D} f(x)$$

NEXT LECTURE

From geometric formulation to planning algorithms

Now we know what a valid path means. Next: how can a computer find one?

Graph search

Represent free space by connected nodes and edges.

Sampling-based

Sample configurations and build candidate connections.

Collision checking

Test states and interpolated motions for validity.

Takeaway

A planner searches in configuration space; every returned path must respect the complete robot.

Summary

- Configuration spaces, workspaces, and some basic ideas about topology
- Types of robots: holonomic/nonholonomic, serial, parallel
- Kinematics and inverse kinematics
- Topology and manifolds
- Relationship between C-space and motion planning